



Desarrollo de Aplicaciones con APIs de IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Introducción a APIs de IA: OpenAI,
Hugging Face, Google AI.



- I. ¿Qué es una API de IA?
- II. Proveedores principales (2025)
- III. Conceptos clave de APIs de IA
- IV. ¿Qué puedes construir con estas APIs?
- V. Comparativa rápida de precios (2025)
- VI. Primer ejemplo: llamada a OpenAI (Node.js)
- VII. Primer ejemplo: llamada a OpenAI (Python)
- VIII. Seguridad: buenas prácticas y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Idea clave

Las APIs de IA permiten incorporar capacidades avanzadas sin necesidad de entrenar, mantener ni desplegar modelos propios. Son la forma más rápida, estable y profesional de añadir inteligencia a cualquier aplicación moderna porque abstraen toda la complejidad del machine learning y ofrecen resultados listos para usar.

—lenguaje, visión, predicción, chat, búsqueda semántica—
Así reducen complejidad, aceleran el desarrollo y permiten incorporar IA profesional en cualquier aplicación con unas pocas líneas de código.

Analogía

En una aplicación se necesita varias habilidades avanzadas:

- entender lenguaje humano,
- reconocer imágenes,
- predecir comportamientos
- mantener conversaciones naturales.

Entrenar tus propios modelos sería como **contratar y formar desde cero a un equipo entero de expertos**: un lingüista, un analista de datos, un experto en visión, un matemático, un psicólogo conversacional... Carísimo, lento y muy difícil de mantener.

Las APIs de IA funcionan como **contratar a esos especialistas ya formados**, disponibles bajo demanda. No necesitas enseñarles nada, ni comprar máquinas, ni entrenar modelos. Solo les envías una petición ("analiza este texto", "resume este documento", "detecta qué hay en esta imagen") y te devuelven la respuesta lista para usar.

Introducción a APIs de IA: OpenAI, Hugging Face, Google AI.



4,1— ¿Qué es una API de IA?

Una API de IA es un servicio en la nube que permite usar inteligencia artificial sin entrenar modelos propios ni disponer de hardware especializado.

Modelos preentrenados Ofrecen capacidades avanzadas listas para usar (texto, visión, audio, predicción).

Respuestas en tiempo real Procesan datos y devuelven resultados en milisegundos, aptos para apps interactivas.

Multicapacidad Una misma API puede analizar texto, clasificar imágenes, transcribir audio o generar contenido.

Escalabilidad y seguridad El proveedor gestiona infraestructura, actualizaciones y protección de datos.

Coste por uso Pagas solo por las llamadas realizadas, sin inversión inicial ni mantenimiento.

Ventaja clave Permiten integrar IA profesional en cualquier aplicación con pocas líneas de código.



4,2— Proveedores principales (2025)



Proveedor	Especialidad	Ideal para	Puntos fuertes
OpenAI	NLP, chat, embeddings	Chatbots, asistentes, generación de texto	Modelos muy potentes y versátiles
Hugging Face	Modelos open-source	Personalización, NLP específico	Comunidad enorme y ecosistema abierto
Google AI (Gemini)	Multimodal, visión	Apps con imágenes + texto	Integración fuerte con visión y búsqueda
Anthropic (Claude)	Contexto largo	Análisis de documentos	Manejo de prompts extensos y razonamiento
AWS Bedrock	Modelos múltiples	Empresas con AWS	Integración nativa con servicios cloud
Azure OpenAI	OpenAI + compliance	Entornos Microsoft	Seguridad, cumplimiento y despliegue empresarial
Cohere	NLP empresarial, embeddings	Búsqueda semántica, clasificación	Rendimiento alto en tareas corporativas
IBM Watsonx	IA empresarial, gobernanza	Sectores regulados	Auditoría, trazabilidad y control del modelo
Mistral AI	Modelos ligeros y eficientes	Integraciones rápidas, despliegues híbridos	Modelos rápidos, económicos y personalizables
Meta (Llama Models)	Modelos open-source potentes	Soluciones on-premise	Libertad total para adaptar y desplegar

OPENAI Es el proveedor más conocido y el que marca el estándar en la industria. Sus modelos GPT-4o y GPT-4o-mini ofrecen el mejor equilibrio entre calidad, velocidad y coste. GPT-4o es multimodal, lo que significa que puede procesar texto, imágenes y audio en la misma conversación. GPT-4o-mini es la opción económica ideal para prototipos y aplicaciones educativas, con un coste hasta 20 veces menor que el modelo completo.

GOOGLE AI

Gemini es la apuesta de Google, con una ventaja competitiva clave: una ventana de contexto de hasta 2 millones de tokens.

Esto significa que puede procesar documentos enormes de una sola vez —por ejemplo, la trilogía completa de El Problema de los Tres Cuerpos (más de 400.000 palabras) en una única petición.

Además, Gemini 1.5 Flash es una versión optimizada para velocidad, ideal para aplicaciones que requieren respuestas en tiempo real.

GOOGLE AI

Gemini es la apuesta de Google, con una ventaja competitiva clave: una ventana de contexto de hasta 2 millones de tokens.

Esto significa que puede procesar documentos enormes de una sola vez —por ejemplo, la trilogía completa de El Problema de los Tres Cuerpos (más de 400.000 palabras) en una única petición.

Además, Gemini 1.5 Flash es una versión optimizada para velocidad, ideal para aplicaciones que requieren respuestas en tiempo real.

ANTHROPIC (CLAUDE)

Anthropic se ha posicionado como la opción preferida para empresas que manejan información sensible.

Claude 3.5 Sonnet destaca en tareas que requieren razonamiento profundo, como análisis de contratos legales, evaluación de código complejo o revisión de documentos financieros.

Su enfoque en "IA constitucional" le permite seguir instrucciones de seguridad con mucha precisión, reduciendo el riesgo de respuestas no deseadas.

HUGGING FACE

A diferencia de los anteriores, Hugging Face no es un único modelo, sino una plataforma que alberga miles de modelos de código abierto. Ofrece dos vías principales: usar modelos gratuitos a través de su API (con límites) o descargar modelos como Mistral, Llama 3 o BLOOM para ejecutarlos en infraestructura propia. Esta segunda opción es ideal cuando se requiere privacidad total, por ejemplo, para procesar datos médicos o información clasificada sin que salga de la organización.

AZURE AI Y AWS BEDROCK

Son las opciones empresariales.

Azure AI ofrece los modelos de OpenAI dentro del ecosistema de Microsoft, con certificaciones de cumplimiento normativo (GDPR, HIPAA, ISO).

AWS Bedrock permite acceder a modelos como Claude, Llama y Titan directamente desde la nube de Amazon, con integración nativa con otros servicios AWS.

```
# EJEMPLO: Comparar respuestas de OpenAI y Gemini (simulado)
# Esto muestra cómo cambiar de proveedor manteniendo la misma lógica
```

```
import os
from openai import OpenAI
import google.generativeai as genai

# ===== CONFIGURACIÓN =====

# OpenAI
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Google Gemini
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
gemini_model = genai.GenerativeModel('gemini-1.5-flash')
```

```
# ===== MISMA PREGUNTA PARA AMBOS =====
pregunta = "Explica qué es una API de IA en una frase"

# Respuesta de OpenAI
respuesta_openai = openai_client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": pregunta}]
).choices[0].message.content

# Respuesta de Gemini
respuesta_gemini = gemini_model.generate_content(pregunta).text

print("=" * 50)
print("OPENAI (GPT-4o-mini):")
print(respuesta_openai)
print("\n" + "=" * 50)
print("GOOGLE GEMINI (1.5 Flash):")
print(respuesta_gemini)
print("=" * 50)
```

```
# ===== HUGGING FACE (modelo gratuito Mistral) =====  
# Nota: Hugging Face requiere token y tiene límites gratuitos  
import requests  
  
API_URL = "https://api-inference.huggingface.co/models/mistralai/Mistral-7B-Instruct-v0.1"  
headers = {"Authorization": f"Bearer {os.getenv('HUGGINGFACE_TOKEN')}}"}  
  
response = requests.post(API_URL, headers=headers, json={"inputs": pregunta})  
respuesta_hf = response.json()[0]["generated_text"]  
  
print("\n" + "=" * 50)  
print("HUGGING FACE (Mistral-7B):")  
print(respuesta_hf[:500]) # Limitar longitud  
print("=" * 50)
```

4.3 — Conceptos clave de APIs de IA

Tokens Son las unidades mínimas que consume un modelo (trozos de palabras). Determinan el coste y la longitud del procesamiento. Más tokens = más coste y más contexto procesado.

Context window Es la cantidad máxima de tokens que el modelo puede "recordar" en una sola interacción. Define el límite de memoria: si lo superas, el modelo olvida partes del mensaje.

Temperature Controla el grado de creatividad. Valores bajos generan respuestas precisas y estables; valores altos producen respuestas más variadas e impredecibles.

4.3 — Conceptos clave de APIs de IA

Max tokens —Es el límite de tokens que el modelo puede generar como salida.

No afecta al input, solo restringe la longitud de la respuesta final.

System prompt —Bloque inicial que define el comportamiento del modelo: rol, estilo, reglas y límites. Actúa como la “configuración global” que guía todas las respuestas posteriores.

Few-shot — ejemplos : Técnica para enseñar al modelo cómo debe responder mostrando ejemplos directamente en el prompt.

Permite ajustar el comportamiento sin entrenar un modelo nuevo.

Embeddings — vectores para búsqueda semántica Representaciones numéricas que capturan el significado de textos o imágenes. Permiten búsquedas por significado, detección de similitud, clasificación y recuperación de información contextual.

4.3 — Conceptos clave de APIs de IA

TOKENS: LA MONEDA DE LAS APIs DE IA

Cuando envías una petición a una API de IA, el coste no se mide en palabras, sino en "tokens". Un token es una unidad de texto que el modelo procesa. En inglés, un token equivale aproximadamente a 4 caracteres o 0.75 palabras. En español, la relación es diferente: un token equivale a unos 1.5 caracteres, por lo que textos en español suelen consumir más tokens que en inglés para la misma cantidad de información.

Para que te hagas una idea:

- Una palabra corta como "hola" = 1 token
- Una palabra larga como "extraordinariamente" = 4 tokens
- Un párrafo de 100 palabras $\approx$ 130-150 tokens
- El Quijote completo (aprox 500.000 palabras) $\approx$ 650.000 tokens

4.3 — Conceptos clave de APIs de IA

CONTEXT WINDOW: LA MEMORIA DEL MODELO

Cada modelo tiene un límite de tokens que puede "recordar" en una sola interacción. Este límite se llama context window.

Si tu conversación supera ese límite, el modelo empieza a olvidar las primeras partes del mensaje.

Imagina que estás manteniendo una conversación con alguien que tiene mala memoria. Si le cuentas una historia larga, cuando llegues al final ya habrá olvidado el principio. Esto es exactamente lo que ocurre con los modelos cuando superas su context window.

Los modelos actuales tienen ventanas de contexto muy variadas:

- GPT-4o-mini: 128.000 tokens (puede procesar un libro de 300 páginas)
- Claude 3.5 Sonnet: 200.000 tokens (unos 500 páginas)
- Gemini 1.5 Pro: 2.000.000 tokens (itoda la trilogía de El Problema de los Tres Cuerpos!)

4.3 — Conceptos clave de APIs de IA

TEMPERATURE: CONTROLANDO LA CREATIVIDAD

La temperatura es un parámetro que va de 0 a 1 (aunque algunos modelos permiten valores superiores) y controla cuán "creativa" o "arriesgada" es la respuesta del modelo.

- Temperatura 0.0-0.3: Respuestas predecibles, consistentes, casi idénticas para la misma entrada. Ideal para tareas que requieren precisión como extraer datos, clasificar textos, generar código o responder preguntas con respuestas objetivas.
- Temperatura 0.4-0.7: Un punto intermedio. Las respuestas varían pero mantienen coherencia. Bueno para asistentes conversacionales donde se busca naturalidad sin perder precisión.
- Temperatura 0.8-1.0: Máxima creatividad. Las respuestas son muy variadas, pueden ser sorprendentes o inesperadas. Ideal para generar ideas creativas, poemas, nombres de productos o cualquier tarea donde la originalidad sea importante.

4.3 — Conceptos clave de APIs de IA

SYSTEM PROMPT: CONFIGURANDO EL COMPORTAMIENTO

El system prompt es un mensaje especial que no ve el usuario final, pero que define cómo debe comportarse el modelo. Es como darle instrucciones previas a un empleado antes de que empiece a atender clientes.

Un buen system prompt debe incluir:

- El rol que debe adoptar (ej: "eres un asistente técnico experto en Python")
- El tono o estilo de comunicación (ej: "responde de forma clara y didáctica")
- Reglas o límites (ej: "no inventes información, si no sabes algo, dilo")
- Formato de respuesta (ej: "estructura tus respuestas con viñetas")

4.3 — Conceptos clave de APIs de IA

FEW-SHOT LEARNING: ENSEÑAR CON EJEMPLOS

El few-shot learning consiste en incluir ejemplos dentro del prompt para que el modelo entienda exactamente qué formato o estilo esperas. Es una forma de "entrenar en caliente" sin necesidad de fine-tuning.

Por ejemplo, si quieres que el modelo clasifique correos como "urgente", "normal" o "spam", puedes mostrarle 2 o 3 ejemplos antes de pedirle que clasifique el correo real.

El modelo aprenderá del patrón que le has mostrado.

4.3 — Conceptos clave de APIs de IA

FEMBEDDINGS: BUSCAR POR SIGNIFICADO

Los embeddings son vectores numéricos que representan el significado de un texto.

Dos textos con significado similar tendrán vectores cercanos en el espacio matemático.

Esto permite hacer búsquedas semánticas: puedes buscar "documentos sobre seguridad informática" y encontrarás textos que hablen de ciberseguridad, incluso si no usan exactamente esas palabras.

Es la tecnología que hay detrás de los motores de búsqueda modernos y los sistemas de recomendación.

```
# EJEMPLO 1: Contar tokens con tiktoken (OpenAI)
import tiktoken

def contar_tokens(texto: str, modelo: str = "gpt-4o"):
    """Cuenta cuántos tokens ocupa un texto en un modelo específico"""
    encoding = tiktoken.encoding_for_model(modelo)
    tokens = encoding.encode(texto)

    print(f"Texto: '{texto[:50]}...'")
    print(f"Caracteres: {len(texto)}")
    print(f"Tokens: {len(tokens)}")
    print(f"Tokens por carácter: {len(tokens)/len(texto):.2f}")
    return tokens

# Probar con diferentes textos
contar_tokens("Hola, ¿cómo estás?")
contar_tokens("La inteligencia artificial está transformando el mundo.")
contar_tokens("Extraordinariamente, el desarrollo de APIs de IA ha democratizado el acceso a modelos avanzados.")

print("\n" + "=" * 50)
```

```
# EJEMPLO 2: Efecto de la temperature
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def probar_temperature(pregunta: str, temperatura: float):
    """Muestra cómo cambia la respuesta según la temperature"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": pregunta}],
        temperature=temperatura
    )
    return response.choices[0].message.content

pregunta = "Escribe una frase sobre la luna"

print("TEMPERATURE 0.0 (predecible):")
print(probar_temperature(pregunta, 0.0))
print("\nTEMPERATURE 0.5 (intermedia):")
print(probar_temperature(pregunta, 0.5))
print("\nTEMPERATURE 1.0 (creativa):")
print(probar_temperature(pregunta, 1.0))

print("\n" + "=" * 50)
```


EJEMPLO 3: System prompt para definir comportamiento

```
def asistente_con_personalidad(pregunta: str, personalidad: str):
```

```
    """Usa system prompt para cambiar el comportamiento del asistente"""
```

```
    system_prompts = {
```

```
        "tecnico": "Eres un experto técnico. Responde con precisión, usando terminología profesional  
y ejemplos de código cuando sea relevante.",
```

```
        "divulgativo": "Eres un divulgador amigable. Explica conceptos complejos de forma sencilla,  
usando analogías y un tono cercano.",
```

```
        "poeta": "Eres un poeta. Responde con lenguaje poético, metafórico y evocador.",
```

```
        "abogado": "Eres un abogado. Responde con formalidad, citando principios y siendo muy cuidadoso con el lenguaje."
```

```
    }
```

```
    response = client.chat.completions.create(
```

```
        model="gpt-4o-mini",
```

```
        messages=[
```

```
            {"role": "system", "content": system_prompts.get(personalidad, system_prompts["divulgativo"])},
```

```
            {"role": "user", "content": pregunta}
```

```
        ],
```

```
        temperature=0.7
```

```
    )
```

```
    return response.choices[0].message.content
```

```
    return response.choices[0].message.content
```

```
print("MISMA PREGUNTA CON DIFERENTES PERSONALIDADES:")
```

```
print("\n🔧 TÉCNICO:")
```

```
print(asistente_con_personalidad("¿Qué es una API?", "tecnico"))
```

```
print("\n📖 DIVULGATIVO:")
```

```
print(asistente_con_personalidad("¿Qué es una API?", "divulgativo"))
```

```
print("\n🎨 POETA:")
```

```
print(asistente_con_personalidad("¿Qué es una API?", "poeta"))
```

```
print("\n" + "=" * 50)
```

```
# EJEMPLO 4: Few-shot learning para clasificación
```

```
def clasificar_con_ejemplos(texto: str):
```

```
    """Usa ejemplos para enseñar al modelo cómo clasificar"""
```

```
    prompt = """
```

```
    Clasifica el sentimiento del texto como "POSITIVO", "NEUTRO" o "NEGATIVO".
```

```
    EJEMPLOS:
```

```
    Texto: "Me encanta este producto, es increíble"
```

```
    Sentimiento: POSITIVO
```

```
    Texto: "El servicio fue aceptable, sin más"
```

```
    Sentimiento: NEUTRO
```

```
    Texto: "Pésima experiencia, no lo recomiendo"
```

```
    Sentimiento: NEGATIVO
```

```
    AHORA CLASIFICA ESTE TEXTO:
```

```
    Texto: """ + texto + """
```

```
    Sentimiento:
```

```
    """
```

```
    response = client.chat.completions.create(
```

```
        model="gpt-4o-mini",
```

```
        messages=[{"role": "user", "content": prompt}],
```

```
        temperature=0.0 # Muy baja para respuestas consistentes
```

```
    )
```

```
    return response.choices[0].message.content.strip()
```

```
# Probar clasificación
```

```
textos = [
```

```
    "La aplicación funciona perfectamente, muy contento",
```

```
    "Regular, cumple pero no esperaba más",
```

```
    "Horrible, perdí mi dinero"
```

```
]
```

```
for texto in textos:
```

```
    resultado = clasificar_con_ejemplos(texto)
```

```
    print(f"'{texto}' → {resultado}")
```

```
print("\n" + "=" * 50)
```

```
# EJEMPLO 5: Embeddings para búsqueda semántica

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def obtener_embedding(texto: str):
    """Obtiene el embedding de un texto usando OpenAI"""
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=texto
    )
    return response.data[0].embedding
```

```
# Crear una base de conocimiento simple

documentos = [
    "La inteligencia artificial permite a las máquinas aprender de la experiencia",
    "Las APIs son interfaces que permiten a programas comunicarse entre sí",
    "El aprendizaje automático es una rama de la inteligencia artificial",
    "Python es un lenguaje de programación muy popular para desarrollo web",
    "Los embeddings convierten texto en vectores numéricos para búsqueda semántica"
]

# Calcular embeddings de todos los documentos
embeddings = [obtener_embedding(doc) for doc in documentos]
```

```

# Calcular embeddings de todos los documentos
embeddings = [obtener_embedding(doc) for doc in documentos]

# Función de búsqueda semántica
def buscar_semanticamente(consulta: str, documentos: list, embeddings: list, top_k: int = 2):
    """Busca documentos por significado, no por palabras exactas"""

    # Embedding de la consulta
    consulta_emb = obtener_embedding(consulta)

    # Calcular similitud con cada documento
    similitudes = []
    for i, doc_emb in enumerate(embeddings):
        sim = cosine_similarity([consulta_emb], [doc_emb])[0][0]
        similitudes.append((i, sim))

# Probar búsqueda semántica
consultas = [
    "¿Cómo aprenden las máquinas?",
    "¿Qué son las interfaces de programación?",
    "Herramientas para convertir texto a números"
]

for consulta in consultas:
    print(f"\n🔍 Consulta: '{consulta}'")
    resultados = buscar_semanticamente(consulta, documentos, embeddings)
    for r in resultados:
        print(f"📄 {r['documento']} (similitud: {r['similitud']:.4f})")

```

```

# Ordenar por similitud descendente
similitudes.sort(key=lambda x: x[1], reverse=True)

# Devolver los top_k más similares
resultados = []
for i, sim in similitudes[:top_k]:
    resultados.append({
        "documento": documentos[i],
        "similitud": sim
    })

return resultados

```

Introducción a APIs de IA: OpenAI, Hugging Face, Google AI.



4.4 — ¿Qué puedes construir con estas APIs?

Chatbots inteligentes Pueden mantener conversaciones naturales, responder dudas, guiar procesos o asistir al usuario dentro de una app.

Ejemplo: un asistente que explica código, resuelve errores o ayuda a navegar una plataforma.

Resumen de documentos Analizan textos largos y generan resúmenes claros, estructurados y adaptados al contexto.

Ejemplo: resumir un PDF de 40 páginas en 5 puntos clave.

Clasificación y análisis de texto Detectan temas, sentimientos, categorías o intenciones dentro de cualquier texto.

Ejemplo: clasificar tickets de soporte según urgencia o tipo de problema.

Introducción a APIs de IA: OpenAI, Hugging Face, Google AI.



4.4 — ¿Qué puedes construir con estas APIs?

Traducción contextual Traducen texto manteniendo el significado, el tono y la intención original.

Ejemplo: traducir documentación técnica sin perder precisión.

Búsqueda semántica Permiten buscar por significado, no por palabras exactas, usando embeddings.

Ejemplo: "encuéntrame documentos que hablen de seguridad en APIs", aunque no usen esas palabras.

Análisis de imágenes Identifican objetos, texto, contenido o patrones dentro de imágenes.

Ejemplo: detectar si una imagen contiene un documento válido o si falta información.



4.4 — ¿Qué puedes construir con estas APIs?

Predicción de datos Anticipan comportamientos basados en patrones: demanda, riesgo, anomalías, clasificación.

Ejemplo: predecir qué usuarios podrían abandonar un servicio.

Aplicaciones multimodales (texto + imagen) Combinan análisis de texto e imágenes en una misma petición.

Ejemplo: "Describe esta imagen y genera un título adecuado para redes sociales".

Introducción a APIs de IA: OpenAI, Hugging Face, Google AI.

4.4 — ¿Qué puedes construir con estas APIs?



Proveedor	Destaca en	Ideal para
OpenAI / Azure OpenAI	Chat, razonamiento, multimodalidad	Chatbots, asistentes, apps conversacionales
Anthropic (Claude)	Contexto largo, análisis profundo	Resumen de documentos, auditorías de texto
Google AI (Gemini)	Visión + texto, traducción	Apps multimodales, análisis de imágenes
Hugging Face	Modelos open-source	Personalización, despliegues propios
Cohere	Embeddings, NLP empresarial	Búsqueda semántica, clasificación
AWS Bedrock	Integración cloud, variedad de modelos	Empresas con infraestructura AWS
IBM Watsonx	Gobernanza, auditoría	Sectores regulados
Mistral AI	Modelos ligeros y rápidos	Integraciones híbridas, on-premise
Meta (Llama)	Open-source potente	Soluciones internas y personalización profunda

Más allá de los ejemplos típicos, las APIs de IA abren un abanico casi infinito de posibilidades.



Aquí exploramos con más detalle cada tipo de aplicación y sus casos de uso reales.

CHATBOTS INTELIGENTES NO SOLO PARA ATENCIÓN AL CLIENTE

Un chatbot con IA no es simplemente un menú de opciones con respuestas predefinidas.

Un chatbot basado en LLM puede mantener conversaciones fluidas, recordar el contexto de lo que se ha dicho antes, adaptar el tono según el usuario y hasta detectar emociones.

Casos de uso reales:

- Asistentes de código que explican errores y sugieren soluciones
- Tutores virtuales que se adaptan al nivel del estudiante
- Entrenadores de ventas que simulan conversaciones con clientes
- Agentes de viajes que planifican itinerarios completos



RESUMEN DE DOCUMENTOS: MÁS ALLÁ DE REDUCIR TEXTO



No se trata solo de acortar un texto.

Un buen resumen con IA puede:

- Extraer puntos clave manteniendo la estructura lógica
- Adaptar el resumen al nivel de conocimiento del lector
- Generar diferentes tipos de resumen (ejecutivo, técnico, divulgativo)
- Identificar información contradictoria dentro del documento

Aplicaciones prácticas:

- Resumir actas de reuniones extrayendo acuerdos y pendientes
- Sintetizar informes médicos para pacientes no especialistas
- Crear resúmenes ejecutivos de contratos legales
- Generar abstracts automáticos de artículos académicos



CLASIFICACIÓN Y ANÁLISIS DE TEXTO A GRAN ESCALA

La IA puede procesar miles de documentos en minutos, clasificándolos por tema, sentimiento, urgencia o cualquier criterio definido.

Esto es especialmente útil para:

- Moderación de contenido: detectar automáticamente mensajes inapropiados
- Análisis de encuestas: categorizar respuestas abiertas
- Triage de tickets de soporte: asignar prioridad y área responsable



TRADUCCIÓN CONTEXTUAL: MÁS QUE PALABRAS



Los modelos actuales no traducen palabra por palabra, sino que entienden el contexto completo.

Esto permite:

- Mantener el tono y estilo del original (formal, coloquial, técnico)
- Adaptar modismos y referencias culturales
- Traducir manteniendo la coherencia en documentos largos
- Realizar traducciones simultáneas en tiempo real



BÚSQUEDA SEMÁNTICA: ENCONTRAR POR SIGNIFICADO

Imagina un buscador que entienda lo que buscas, no solo las palabras que usas.

Con embeddings puedes:

- Buscar en miles de documentos por concepto, no por keyword
- Recomendar contenido similar basado en significado
- Detectar plagio o contenido duplicado
- Agrupar documentos por temas sin etiquetas manuales



ANÁLISIS DE IMÁGENES: VISIÓN POR COMPUTADOR dicampus

SIMPLIFICADA

Los modelos multimodales como GPT-4o o Gemini pueden "ver" imágenes y describir su contenido, detectar objetos, leer texto dentro de imágenes e incluso identificar patrones visuales complejos.

Aplicaciones:

- Validación de documentos: comprobar si una imagen contiene un DNI válido
- Diagnóstico asistido: analizar radiografías o imágenes médicas
- Accesibilidad: describir imágenes para personas con discapacidad visual
- Moderación visual: detectar contenido inapropiado en imágenes



PREDICCIÓN DE DATOS: ANTICIPAR EL FUTURO



Aunque no son modelos específicos de forecasting, los LLM pueden analizar patrones en datos históricos y hacer predicciones razonables sobre tendencias, comportamientos o riesgos.

Casos prácticos:

- Predecir qué clientes tienen mayor riesgo de abandonar
- Anticipar picos de demanda en base a patrones históricos
- Identificar transacciones sospechosas por comportamiento anómalo



APLICACIONES MULTIMODALES: COMBINANDO TEXTO, IMAGEN Y AUDIO



La verdadera potencia surge cuando combinas diferentes capacidades.

Por ejemplo:

- Un sistema que recibe una foto de un producto, extrae su texto, lo traduce, genera una descripción y la publica en redes sociales
- Un asistente que escucha una conversación, la transcribe, identifica los temas principales y genera un resumen con tareas pendientes
- Una herramienta educativa que analiza el dibujo de un estudiante, entiende qué representa y ofrece retroalimentación personalizada




```
# EJEMPLO 1: Chatbot con memoria de conversación
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

```
class ChatbotConMemoria:
    """Chatbot que mantiene el historial de conversación"""
```

```
    def __init__(self, sistema_instrucciones: str = None):
        self.historial = []
        if sistema_instrucciones:
            self.historial.append({"role": "system", "content": sistema_instrucciones})
```

```
    def preguntar(self, mensaje: str) -> str:
        """Envía un mensaje y guarda la respuesta en el historial"""
        # Añadir mensaje del usuario
        self.historial.append({"role": "user", "content": mensaje})
```

```
        # Obtener respuesta
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=self.historial
        )
```

```
        respuesta = response.choices[0].message.content
```

```
        # Guardar respuesta en historial
        self.historial.append({"role": "assistant", "content": respuesta})
```

```
        return respuesta
```

```
    def resetear(self):
        """Reinicia la conversación"""
        sistema = self.historial[0] if self.historial and self.historial[0]["role"] == "system" else
None
        self.historial = [sistema] if sistema else []
```

```
# Crear chatbot con personalidad
bot = ChatbotConMemoria(
    sistema_instrucciones="Eres un tutor de programación. Explica conceptos de forma didáctica, usando analogías y ejemplos prácticos."
)
```

```
print("=== CHATBOT TUTOR DE PROGRAMACIÓN ===")
respuesta = bot.preguntar("¿Qué es una variable?")
print(f"Usuario: ¿Qué es una variable?")
print(f"Tutor: {respuesta}\n")
```

```
respuesta = bot.preguntar("¿Y cómo se declara en Python?")
print(f"Usuario: ¿Y cómo se declara en Python?")
print(f"Tutor: {respuesta}\n")
```

```
print("\n" + "=" * 70)
```

```
# EJEMPLO 2: Resumen de documentos con diferentes niveles de detalle
```

```
def resumir_documento(texto: str, nivel: str = "ejecutivo") -> str:
```

```
    """
```

```
    Genera resúmenes con diferentes niveles de detalle.
```

```
    niveles:
```

- ejecutivo: muy breve, solo la idea principal
- técnico: incluye detalles y estructura
- didáctico: explicación paso a paso

```
    """
```

```
    instrucciones = {
```

```
        "ejecutivo": "Genera un resumen ejecutivo de máximo 3 líneas. Solo los puntos más importantes.",
```

```
        "tecnico": "Genera un resumen técnico estructurado con secciones. Incluye detalles relevantes.",
```

```
        "didactico": "Genera una explicación didáctica, como si se lo enseñaras a un principiante. Usa analogías."
```

```
    }
```

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": instrucciones.get(nivel, instrucciones["ejecutivo"])},
        {"role": "user", "content": f"Resume esto:\n\n{texto}"}
    ],
    max_tokens=500
)

return response.choices[0].message.content
```

Documento de ejemplo

```
documento_largo = """
```

La inteligencia artificial es un campo de la informática que se centra en crear sistemas capaces de realizar tareas

que normalmente requieren inteligencia humana. Esto incluye el aprendizaje, el razonamiento, la percepción y la

comprensión del lenguaje natural. En los últimos años, el desarrollo de modelos de lenguaje de gran tamaño ha

democratizado el acceso a la IA, permitiendo que cualquier desarrollador pueda integrar capacidades avanzadas en

sus aplicaciones mediante APIs. Estos modelos son entrenados con cantidades masivas de datos y pueden realizar tareas

como generar texto, responder preguntas, traducir idiomas, clasificar contenido y mucho más. La principal ventaja

de usar APIs de IA es que abstraen toda la complejidad del entrenamiento y despliegue, permitiendo a los

desarrolladores centrarse en la lógica de negocio de sus aplicaciones.

```
"""
```

```
print("=== RESUMEN EJECUTIVO ===")
print(resumir_documento(documento_largo, "ejecutivo"))
print("\n=== RESUMEN TÉCNICO ===")
print(resumir_documento(documento_largo, "tecnico"))
print("\n=== EXPLICACIÓN DIDÁCTICA ===")
print(resumir_documento(documento_largo, "didactico"))
```

```
print("\n" + "=" * 70)
```

```
# EJEMPLO 3: Clasificación de tickets de soporte
def clasificar_ticket(descripcion: str) -> dict:
    """Clasifica un ticket de soporte en categoría, prioridad y departamento"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": """
            Eres un sistema de clasificación de tickets de soporte.

            Responde ÚNICAMENTE en formato JSON con estos campos:
            - categoria: una de [tecnico, facturacion, cuenta, producto, otro]
            - prioridad: una de [alta, media, baja]
            - departamento: una de [soporte, facturacion, desarrollo, ventas]
            - descripcion_corta: resumen de una línea del problema

            No añadas texto fuera del JSON.
            """},
            {"role": "user", "content": descripcion}
        ],
        temperature=0.0
    )
```

```
import json
try:
    return json.loads(response.choices[0].message.content)
except:
    return {"error": "No se pudo clasificar"}

tickets = [
    "No puedo iniciar sesión, me dice que la contraseña es incorrecta aunque la acabo de cambiar",
    "Me cobraron dos veces la suscripción de este mes, necesito que me devuelvan el dinero",
    "La aplicación se cierra cuando intento subir una foto muy grande"
]

for ticket in tickets:
    clasificacion = clasificar_ticket(ticket)
    print(f"\n🎫 Ticket: {ticket[:60]}...")
    print(f"📌 Categoría: {clasificacion.get('categoria')}")
    print(f"⚠️ Prioridad: {clasificacion.get('prioridad')}")
    print(f"🏢 Departamento: {clasificacion.get('departamento')}")

print("\n" + "=" * 70)
```

```
# EJEMPLO 4: Análisis de imagen (multimodal con GPT-4o)

import base64
from PIL import Image

def analizar_imagen(ruta_imagen: str, pregunta: str = "¿Qué hay en esta imagen? Describe en detalle.") -> str:
    """
    Analiza una imagen usando GPT-4o (requiere tener acceso a gpt-4o)
    """

    # Leer imagen y convertir a base64
    with open(ruta_imagen, "rb") as f:
        imagen_base64 = base64.b64encode(f.read()).decode("utf-8")

    response = client.chat.completions.create(
        model="gpt-4o", # Necesita GPT-4o, no funciona con mini
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": pregunta},
                    {
                        "type": "image_url",
                        "image_url": {"url": f"data:image/jpeg;base64,{imagen_base64}"}
                    }
                ]
            }
        ],
        max_tokens=500
    )
```

```
# Nota: Este código requiere una imagen real y acceso a GPT-4o
# print(analizar_imagen("foto.jpg", "¿Qué objetos hay en esta imagen?"))

print("\n" + "=" * 70)
```

EJEMPLO 5: Traducción contextual con mantenimiento de estilo

```
def traducir_con_estilo(texto: str, idioma_destino: str, estilo: str = "formal") -> str:
```

```
    """
```

```
    Traduce manteniendo el estilo y tono del original.
```

```
    estilos: formal, coloquial, técnico, poético
```

```
    """
```

```
    estilos_descripcion = {
```

```
        "formal": "manteniendo un tono profesional y formal",
```

```
        "coloquial": "usando un lenguaje cotidiano y cercano",
```

```
        "tecnico": "conservando la terminología técnica precisa",
```

```
        "poetico": "utilizando un lenguaje literario y evocador"
```

```
    }
```

```
    response = client.chat.completions.create(
```

```
        model="gpt-4o-mini",
```

```
        messages=[
```

```
            {"role": "system", "content": f"""
```

```
            Traduce el siguiente texto al {idioma_destino}, {estilos_descripcion.get(estilo, 'forma
```

```
            l')}}.
```

```
            Mantén el significado original, el tono y cualquier matiz cultural.
```

```
            Devuelve SOLO la traducción, sin comentarios adicionales.
```

```
            """,
```

```
            {"role": "user", "content": texto}
```

```
        ],
```

```
        temperature=0.3
```

```
    )
```

```
    return response.choices[0].message.content
```

```
texto_original = "Hey, I'm really sorry but I won't be able to make it to the meeting. Something urgent came up. Let's catch up later this week if you're free!"
```

```
print("=== TRADUCCIONES CON DIFERENTES ESTILOS ===")
```

```
print(f"Original: {texto_original}\n")
```

```
print(f"Formal (español): {traducir_con_estilo(texto_original, 'español', 'formal')}")
```

```
print(f"Coloquial (español): {traducir_con_estilo(texto_original, 'español', 'coloquial')}")
```

```
print("\n" + "=" * 70)
```



4.5 — Comparativa rápida de precios (2025)

(Precios aproximados por 1K tokens o equivalente. Pensado para comparación, no para facturación exacta.)



API / Modelo	Precio base (2025)	Comentario técnico
OpenAI GPT-4o-mini	\$0.002 / 1K tokens	Muy económico, rápido y suficientemente potente para la mayoría de apps.
OpenAI GPT-4o	\$0.005-0.01 / 1K tokens	Mejor razonamiento y multimodalidad; ideal para chatbots avanzados.
Claude 3 (Haiku)	\$0.25 / 1M tokens	Muy barato y rápido; excelente para tareas masivas.
Claude 3 (Sonnet)	\$3 / 1M tokens	Ideal para documentos largos y análisis profundo.
Claude 3 (Opus)	\$15 / 1M tokens	Máxima calidad de razonamiento; uso profesional.
Google Gemini 1.5 Flash	Gratis (límite) / muy bajo coste	Excelente en visión y multimodalidad ligera.
Google Gemini 1.5 Pro	\$0.0025-0.01 / 1K tokens	Muy fuerte en visión + texto; buena alternativa a GPT-4o.
Hugging Face Inference API	Gratis (modelos open-source)	Ideal para prototipos y despliegues personalizados.
Cohere Command R	\$0.5-\$1 / 1M tokens	Muy eficiente en embeddings y RAG empresarial.
Mistral (API oficial)	\$0.002-0.004 / 1K tokens	Modelos rápidos, ligeros y económicos.
Meta Llama (vía proveedores)	Variable / muy bajo	Ideal para despliegues on-premise o privados.
AWS Bedrock	Depende del modelo	Pago por uso; integración nativa con AWS.
Azure OpenAI	Igual que OpenAI + coste Azure	Añade compliance, seguridad y despliegue empresarial.

■ 4.5 — Comparativa rápida de precios (2025)

- GPT-4o-mini es el mejor equilibrio precio/rendimiento para apps educativas y prototipos.
- Claude 3 Sonnet domina en documentos largos y análisis profundo.
- Gemini destaca en visión y multimodalidad.
- Hugging Face / Mistral / Llama son perfectos para open-source, on-premise o proyectos con privacidad estricta.
- Azure y AWS son la opción natural para empresas ya integradas en esos ecosistemas.

4.5 — Comparativa rápida de precios (2025)

- CÓMO CALCULAR EL COSTE DE UNA APLICACIÓN REAL
-
- Imagina que construyes un chatbot educativo que:
- - Recibe 100 consultas por día
- - Cada consulta tiene 200 tokens de entrada (la pregunta + contexto)
- - Cada respuesta genera 300 tokens de salida
-
- Coste diario = $(100 \times 200 \times \$0.15 / 1.000.000) + (100 \times 300 \times \$0.60 / 1.000.000)$
- = $(\$3.00) + (\$18.00) / 1.000?$

4.5 — Comparativa rápida de precios (2025)

- Cálculo correcto:
- - Input diario: $100 \times 200 = 20.000$ tokens $\rightarrow 20.000 / 1.000.000 \times \$0.15 = \$0.003$
- - Output diario: $100 \times 300 = 30.000$ tokens $\rightarrow 30.000 / 1.000.000 \times \$0.60 = \$0.018$
- - Coste diario total: \$0.021 (2 céntimos de dólar)
- - Coste mensual: \$0.63 (menos de 1 dólar)
-
- ¡Esto es increíblemente económico!
- Pero cuidado, si escalas a 10.000 consultas diarias, el coste mensual sería de unos \$63.

■ 4.5 — Comparativa rápida de precios (2025)

- ESTRATEGIAS PARA OPTIMIZAR COSTES
- 1. Elegir el modelo adecuado para cada tarea
 - - GPT-4o-mini para tareas simples (clasificación, resúmenes cortos)
 - - GPT-4o solo cuando se necesita razonamiento complejo o multimodalidad
- 2. Limitar la longitud de respuestas con max_tokens
 - - No generar 1000 tokens si con 200 es suficiente
- 3. Cachear respuestas comunes
 - - Si la misma pregunta se repite, guarda la respuesta
- 4. Comprimir prompts
 - - Reduce espacios, usa nombres cortos en ejemplos
- 5. Establecer límites mensuales en la consola del proveedor
- 6. Monitorizar consumo con dashboards y alertas

■ 4.5 — Comparativa rápida de precios (2025)

- ESTRATEGIAS PARA OPTIMIZAR COSTES
- 1. Elegir el modelo adecuado para cada tarea
 - - GPT-4o-mini para tareas simples (clasificación, resúmenes cortos)
 - - GPT-4o solo cuando se necesita razonamiento complejo o multimodalidad
- 2. Limitar la longitud de respuestas con max_tokens
 - - No generar 1000 tokens si con 200 es suficiente
- 3. Cachear respuestas comunes
 - - Si la misma pregunta se repite, guarda la respuesta
- 4. Comprimir prompts
 - - Reduce espacios, usa nombres cortos en ejemplos
- 5. Establecer límites mensuales en la consola del proveedor
- 6. Monitorizar consumo con dashboards y alertas

■ 4.5 — Comparativa rápida de precios (2025)

Tarea	Tokens aprox	GPT-4o-mini	GPT-4o	Gemini Flash
Clasificar un ticket	200 input + 50 output	\$0.00006	\$0.0009	\$0.00003
Resumir artículo (1000 palabras)	1500 input + 300 output	\$0.00040	\$0.0060	\$0.00020
Traducir documento (5000 palabras)	7500 input + 6500 output	\$0.0050	\$0.0750	\$0.0025
Analizar imagen	1000 tokens imagen + 300 output	N/A	\$0.0050	\$0.0030
Libro completo (300 páginas)	100.000 input + 5.000 output	\$0.045	\$0.75	\$0.021

■ 4.5 — Comparativa rápida de precios (2025)

Las APIs de IA son extremadamente económicas para la mayoría de casos de uso.

Una aplicación con miles de usuarios diarios puede costar menos que un café al mes.

El verdadero reto no es el coste por llamada, sino controlar el uso descontrolado (bucles infinitos, ataques, malas configuraciones) que pueden disparar la factura.

■ 4.5 — Comparativa rápida de precios (2025)

Las APIs de IA son extremadamente económicas para la mayoría de casos de uso.

Una aplicación con miles de usuarios diarios puede costar menos que un café al mes.

El verdadero reto no es el coste por llamada, sino controlar el uso descontrolado (bucles infinitos, ataques, malas configuraciones) que pueden disparar la factura.

4.6 — Primer ejemplo: llamada a OpenAI (Node.js)

```
import OpenAI from "openai";

const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

const response = await client.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: "¿Qué es una API de IA?" }]
});

console.log(response.choices[0].message.content);
```

¿Qué está ocurriendo?

¿Por qué es importante esta estructura?

4.6 — Primer ejemplo: llamada a OpenAI (Node.js)



¿Qué está ocurriendo?

```
client.chat.completions.create({...})
```

Es la función que envía una petición al modelo de lenguaje. En este caso, se está usando el endpoint de **chat completions**, diseñado para mantener conversaciones tipo chat.

```
model: "gpt-4o-mini"
```

Indica qué modelo se va a usar. *GPT-4o-mini* es una versión ligera, rápida y económica del modelo GPT-4o, ideal para tareas cotidianas como responder preguntas, generar texto o asistir en aplicaciones educativas.

```
messages: [{ role: "user", content: "¿Qué es una API de IA?" }]
```

Define el mensaje que se envía al modelo.

"role: user" indica que el mensaje viene del usuario.

"content" contiene la pregunta que queremos que la IA responda. Este formato permite mantener el historial de conversación si se añaden más mensajes (por ejemplo, del sistema o del asistente).

```
await
```

La llamada es asíncrona: el código espera la respuesta del modelo antes de continuar. Esto es clave para integrarlo en aplicaciones que dependen de la respuesta generada.

```
response.choices[0].message.content
```

Extrae el texto generado por el modelo. Es lo que se muestra al usuario como respuesta.

```
const response = await client.chat.completions.create({  
  model: "gpt-4o-mini",  
  messages: [{ role: "user", content: "¿Qué es una API de IA?" }]  
});
```



4.6 — Primer ejemplo: llamada a OpenAI (Node.js)



¿Por qué es importante esta estructura?

- **Permite modularidad:** puedes cambiar el modelo, el mensaje o el rol sin modificar el resto del código.
- **Es escalable:** puedes añadir mensajes anteriores para mantener el contexto.
- **Es compatible** con cualquier aplicación web, móvil o backend que use JavaScript.



4.6 — Primer ejemplo: llamada a OpenAI (Node.js)

```
import OpenAI from "openai";

const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

const response = await client.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: "¿Qué es una API de IA?" }]
});

console.log(response.choices[0].message.content);
```

¿Qué está ocurriendo?

¿Por qué es importante esta estructura?

4.7 — Primer ejemplo: llamada a OpenAI (Python)

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "¿Qué es una API de IA?"}]
)

print(response.choices[0].message.content)
```

¿Qué está ocurriendo?

¿Por qué es importante esta estructura?



4.7 — Primer ejemplo: llamada a OpenAI (Python)



¿Qué está ocurriendo?

```
client.chat.completions.create(...)
```

Es la función que envía una petición al modelo de lenguaje. Se usa el endpoint de **chat completions**, diseñado para conversaciones tipo chat, donde se puede mantener contexto entre mensajes.

```
model="gpt-4o-mini"
```

Indica qué modelo se va a usar. *GPT-4o-mini* es una versión ligera y económica del modelo GPT-4o, ideal para tareas rápidas como responder preguntas, generar texto o asistir en aplicaciones educativas.

```
messages=[{"role": "user", "content": "¿Qué es una API de IA?"}]
```

Define el mensaje que se envía al modelo.

"role": "user" indica que el mensaje proviene del usuario.

"content" contiene la pregunta que queremos que la IA responda.

Este formato permite construir conversaciones más largas si se añaden mensajes anteriores del sistema o del asistente.

```
response.choices[0].message.content
```

Extrae el texto generado por el modelo. Es la respuesta que se imprime en consola y que puede usarse en la aplicación.

```
response = client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=[{"role": "user", "content": "¿Qué es una API de IA?"}]  
)
```



Llamada a la API de OpenAI (JavaScript vs Python)



Aspecto	JavaScript	Python
Cliente usado	OpenAI desde el paquete oficial	OpenAI desde el paquete oficial
Inicialización	<code>new OpenAI({ apiKey })</code>	<code>OpenAI(api_key=...)</code>
Gestión de entorno	<code>process.env.OPENAI_API_KEY</code>	<code>os.getenv("OPENAI_API_KEY")</code>
Formato de llamada	<code>await client.chat.completions.create({...})</code>	<code>client.chat.completions.create(...)</code>
Asincronía	Requiere <code>await</code> y entorno <code>async</code>	Llamada directa (bloqueante por defecto)
Extracción de respuesta	<code>response.choices[0].message.content</code>	<code>response.choices[0].message.content</code>
Lenguaje del prompt	Español: <code>"¿Qué es una API de IA?"</code>	Español: <code>"¿Qué es una API de IA?"</code>
Modelo usado	<code>"gpt-4o-mini"</code>	<code>"gpt-4o-mini"</code>
Entorno típico	Frontend, Node.js, apps web	Backend, scripts, pipelines, bots

■ ¿Qué diferencias prácticas hay?

- **JavaScript** se usa más en entornos web y requiere manejo explícito de asincronía (`await`, `async`). Ideal para integraciones en apps interactivas.
- **Python** es más directo y se adapta mejor a scripts, automatizaciones y entornos backend, no necesita `await` por defecto.
- Ambos usan el mismo endpoint (`chat.completions`) y devuelven la respuesta en el mismo formato.

MÁS ALLÁ DEL EJEMPLO BÁSICO: CÓDIGO LISTO PARA PRODUCCIÓN

Los ejemplos anteriores muestran la llamada básica a la API, pero una aplicación real necesita manejar errores, gestionar configuraciones, monitorizar costes y respetar buenas prácticas de seguridad.

```
mi-app-ia/  
├── .env                # Variables de entorno (NUNCA en GitHub)  
├── .gitignore          # Evita subir .env, claves, etc.  
├── requirements.txt     # Dependencias de Python  
├── config.py           # Configuración centralizada  
├── monitor.py          # Monitorización de costes  
├── cliente_ia.py       # Cliente wrapper de la API  
└── app.py              # Lógica principal
```


MANEJO DE ERRORES ROBUSTO

Las APIs de IA pueden fallar por múltiples razones:

- Límites de tasa (rate limits): has hecho demasiadas peticiones
- Timeouts: el servidor tarda más de lo esperado
- Errores de autenticación: API key inválida o expirada
- Errores de validación: el prompt excede el context window
- Errores del servidor (5xx): problemas en el proveedor

Un buen código debe manejar todos estos casos con reintentos y fallbacks.

MONITORIZACIÓN DE COSTES



Cada llamada consume tokens que cuestan dinero. Es fundamental:

- Registrar cuántos tokens se usan por petición
- Establecer límites diarios o mensuales
- Alertar cuando se superan umbrales
- Analizar qué usuarios o funciones consumen más

CONFIGURACIÓN POR ENTORNOS

Debes poder usar diferentes modelos y límites según el entorno:

- Desarrollo: modelo barato (GPT-4o-mini), sin límites estrictos
- Testing: mismo modelo que producción pero con datos de prueba
- Producción: modelo completo, con límites y monitorización



Nunca, bajo ninguna circunstancia, incluyas API keys  dicampus directamente en el código.

Utiliza variables de entorno cargadas desde un archivo `.env` que esté en `.gitignore`.

Ejemplo de archivo `.env`:

```
OPENAI_API_KEY=sk-proj-abc123...
```

```
OPENAI_ORG_ID=org-xyz789...
```

```
ENV=production
```

```
MAX_TOKENS_POR_DEFECTO=500
```

```
MODELO_POR_DEFECTO=gpt-4o-mini
```

```
LIMITE_TOKENS_DIARIO=100000
```



■ 4.8 — Seguridad: buenas prácticas y errores comunes



Nunca hardcodear API keys Las claves no deben aparecer en el código fuente ni en repositorios. Evita exponerlas en GitHub, capturas o logs.

Usar variables de entorno Las keys deben cargarse desde el sistema operativo o un gestor seguro (dotenv, secrets manager). Mantiene el código limpio y seguro.

Añadir .env al .gitignore Evita que los archivos con claves o configuraciones sensibles se suban al repositorio.

Rotar keys periódicamente Cambiar las claves reduce el impacto si alguna se filtra. Es una práctica estándar en seguridad.

Limitar uso por entorno (dev/staging/prod) Cada entorno debe tener su propia key con permisos y límites distintos. Minimiza daños en caso de abuso.

Monitorizar consumo y costos Revisar dashboards y alertas evita sorpresas en facturación y detecta usos anómalos.

Implementar rate limiting Protege tu API de abusos, bucles infinitos o ataques. Evita que un error dispare miles de llamadas.



■ 4.8 — Seguridad: buenas prácticas y errores comunes



No proteger las API keys Suben claves a GitHub, las dejan en el código o las comparten sin querer. Es el fallo más frecuente.

No controlar costos No ponen límites, no monitorizan y usan modelos caros sin necesidad. Resultado: facturas inesperadas.

Usar modelos demasiado grandes Eligen modelos "top" para tareas simples. Consumen más tokens, más coste y más latencia sin aportar valor.

No entender tokens ni límites No calculan el coste por token, no conocen el context window y saturan el modelo con prompts enormes.

No manejar errores de red Asumen que la API siempre responde. Falta de try/except, reintentos o gestión de timeouts.

No validar respuestas de la IA Confían ciegamente en la salida del modelo. No comprueban formato, contenido ni coherencia antes de usarlo.





Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



